

TUGAS
SISTEM OPERASI



Disusun oleh :
Muhammad Rafi Hendardi
2410651062

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

DAFTAR ISI

TUGAS	1
SISTEM OPERASI	1
DAFTAR ISI.....	2
BAB III SOAL PEMROGRAMAN.....	3
3.1. Analisis Masalah	3
3.2. Implementasi.....	3
3.3. Testing Output	4
3.4. Analisis Hasil.....	6

BAB III

SOAL PEMROGRAMAN

3.1. Analisis Masalah

- **Pemahaman Soal**
Masalah dining philosophers terdapat lima filsuf yang harus bersaing untuk mendapatkan sumpit yang terbatas agar dapat makan.
- **Identifikasi masalah sinkronisasi (deadlock)**
Deadlock terjadi Ketika setiap filsuf mengambil sumpit kiri secara Bersama-sama dan kemudian menunggu sumput kanannya, sehingga menciptakan rantai tunggu yang tak terputus.
 - **Mutual Exclusion** (Dieliminasi oleh mekanisme *semaphore* pada sumpit).
 - **Hold and Wait** (Dieliminasi jika filsuf harus mengambil kedua sumpit sekaligus atau tidak sama sekali).
 - **No Preemption** (Tidak dapat diambil paksa).
 - **Circular Wait** (Rantai tunggu melingkar. Inilah yang utama harus dipecah).
- **Desain Solusi**

Skenario	Tujuan	Prinsip Pencegahan	Kondisi dihilangkan
Demonstrasi	Menimbulkan deadlock	Ambil sumpit kiri, lalu kanan (selalu).	Circular wait dipertahankan.
Solusi A	Pencegahan deadlock	Resource ordering (filsuf genap/ganjil memiliki urutan pengambilan berbeda).	Circular wait
Solusi B	Pencegahan deadlock	Limiting Philosophers (Maksimal N-1 filsuf diizinkan mencoba makan).	Hold and wait / Circular Wait

3.2. Implementasi

- **Penjelasan Algoritma**
 - **Solusi A**
 1. Filsuf genap (0, 2, 4) : Selalu mengambil sumpit kiri lalu sumpit kanan. Ini urutan sumber daya dari kecil ke besar.

2. Filsuf ganjil (1, 3) : Selalu mencoba mengambil sumpit kanan lalu sumpit kiri. Ini urutan sumber daya dari besar ke kecil.
3. Rantai tunggu melingkar terputus karena satu filsuf selalu mau mengambil sumpit terbesar yang mungkin dibutuhkan oleh filsuf lain.

➤ **Solusi B**

1. Didefinisikan satu *semaphore* tambahan, **room**, dengan nilai awal N-1 (yaitu 4).
2. Seorang filsuf harus melakukan **sem_wait(&room)** untuk masuk ke "ruangan makan" sebelum mencoba mengambil sumpit.
3. Di dalam ruangan, filsuf mengambil sumpit kiri dan kanan secara berurutan.
4. Setelah selesai makan, filsuf melakukan **sem_post(&room)** untuk memberi izin filsuf lain masuk.

• Potongan Kode Penting dan Penjelasan

Solusi	Potongan Kode	Penjelasan
A (Ordering)	<pre>If (i % 2 == 0) { first = left_c; second = right_c; } else { first = right_c; second = left_c; }</pre>	Menetapkan urutan pengambilan sumpit yang berbeda untuk filsuf ganjil/genap untuk memecah Circular Wait.
B (Limiting)	<pre>sem_t room; sem_init(&room, 0, N - 1);</pre>	Inisialisasi Monitor atau room semaphore dengan nilai 4, membatasi jumlah proses aktif yang rentan deadlock.
B (Limiting)	<pre>sem_wait(&room); // ... ambil sumpit ... sem_post(&room);</pre>	Menghilangkan potensi deadlock dengan memastikan hanya \$N-1\$ filsuf yang bisa berada dalam kondisi Hold and Wait.

• Mengapa Memilih Mekanisme tertentu

Memilih semaphore karena memiliki mekanisme sinkronisasi yang paling umum dan lugas untuk mengontrol akses ke sumber daya terbatas (sumpit) dan untuk mengimplementasikan monitor (Solusi B)

3.3. Testing Output

- Test case yang dicoba
 1. Deadlock Demonstration: Uji coba dijalankan hingga *hang* (Deadlock) terbukti.
 2. Solusi A (Resource Ordering): Dijalankan selama 60 detik untuk memverifikasi fungsionalitas dan mengukur efisiensi.

3. Solusi B (Limiting Philosophers): Dijalankan selama 60 detik untuk memverifikasi fungsionalitas dan mengukur efisiensi.
- Screenshot Output dengan berbagai Skenario
 - Skenario 1 : Deadlock Demonstration

```

rafi@DESKTOP-TM81UKJ:~/Tugas5/deadlock_demonstration$ nano deadlock_demonstration.c
rafi@DESKTOP-TM81UKJ:~/Tugas5/deadlock_demonstration$ gcc deadlock_demonstration.c -o deadlock_demonstration
rafi@DESKTOP-TM81UKJ:~/Tugas5/deadlock_demonstration$ ./deadlock_demonstration
=== DEMONSTRASI DEADLOCK (Dining Philosophers) ===
Filsuf 0 sedang berpikir...
Filsuf 1 sedang berpikir...
Filsuf 2 sedang berpikir...
Filsuf 3 sedang berpikir...
Filsuf 4 sedang berpikir...
Filsuf 2 mencoba mengambil sumpit kiri 2
Filsuf 2 berhasil mengambil sumpit kiri 2
Filsuf 1 mencoba mengambil sumpit kiri 1
Filsuf 1 berhasil mengambil sumpit kiri 1
Filsuf 3 mencoba mengambil sumpit kiri 3
Filsuf 4 mencoba mengambil sumpit kiri 4
Filsuf 4 berhasil mengambil sumpit kiri 4
Filsuf 3 berhasil mengambil sumpit kiri 3
Filsuf 0 mencoba mengambil sumpit kiri 0
Filsuf 0 berhasil mengambil sumpit kiri 0
Filsuf 2 mencoba mengambil sumpit kanan 3
Filsuf 1 mencoba mengambil sumpit kanan 2
Filsuf 4 mencoba mengambil sumpit kanan 0
Filsuf 3 mencoba mengambil sumpit kanan 4
Filsuf 0 mencoba mengambil sumpit kanan 1

```

- Skenario 2 : Solusi A (Resource Ordering)

```

[00:59] [0] Filsuf-5 sedang berpikir...
[00:59] [0] Filsuf-4 mengambil sumpit kedua...
[00:59] [0] Filsuf-4 sedang MAKAN...
[01:00] [00] Filsuf-2 menaruh kedua sumpit
[01:00] [0] Filsuf-3 mengambil sumpit kedua...
[01:00] [0] Filsuf-1 lapar, mencoba ambil sumpit...
[01:00] [0] Filsuf-1 mengambil sumpit pertama...
[01:00] [0] Filsuf-1 mengambil sumpit kedua...
[01:00] [0] Filsuf-1 sedang MAKAN...
[01:00] [00] Filsuf-4 menaruh kedua sumpit
[01:00] [0] Filsuf-3 sedang MAKAN...
[01:01] [0] Filsuf-5 lapar, mencoba ambil sumpit...
[01:01] [0] Filsuf-5 mengambil sumpit pertama...
[01:01] [0] Filsuf-5 mengambil sumpit kedua...
[01:01] [00] Filsuf-1 menaruh kedua sumpit
[01:01] [0] Filsuf-5 sedang MAKAN...
[01:02] [00] Filsuf-5 menaruh kedua sumpit
[01:02] [00] Filsuf-3 menaruh kedua sumpit

=== STATISTIK (60 detik) ===
Filsuf-1: Makan 25 kali
Filsuf-2: Makan 23 kali
Filsuf-3: Makan 22 kali
Filsuf-4: Makan 21 kali
Filsuf-5: Makan 22 kali

[00] Total berhasil makan: 113 kali
[00] Tidak ada deadlock! (Program berjalan hingga selesai)
[00] Distribusi fairness: CUKUP/KURANG (Rentang 4)
rafi@DESKTOP-TM81UKJ:~/Tugas5/deadlock_prevention$

```

- Skenario 3 : Solusi B (Limiting Philosophers)

```

rafi@DESKTOP-TM81UKJ: ~/Tugas5/deadlock_prevention
[01:00] [00] Filsuf-5 menaruh kedua sumpit
[01:00] [00] Filsuf-5 keluar ruangan
[01:00] [00] Filsuf-4 sedang MAKAN...
[01:00] [00] Filsuf-1 lapar, mencoba masuk ruangan...
[01:00] [00] Filsuf-1 berhasil masuk ruangan...
[01:00] [00] Filsuf-1 mengambil sumpit kiri...
[01:00] [00] Filsuf-1 mengambil sumpit kanan...
[01:00] [00] Filsuf-2 menaruh kedua sumpit
[01:00] [00] Filsuf-2 keluar ruangan
[01:00] [00] Filsuf-3 mengambil sumpit kanan...
[01:00] [00] Filsuf-1 sedang MAKAN...
[01:01] [00] Filsuf-4 menaruh kedua sumpit
[01:01] [00] Filsuf-4 keluar ruangan
[01:01] [00] Filsuf-3 sedang MAKAN...
[01:01] [00] Filsuf-1 menaruh kedua sumpit
[01:01] [00] Filsuf-1 keluar ruangan
[01:02] [00] Filsuf-3 menaruh kedua sumpit
[01:02] [00] Filsuf-3 keluar ruangan

=== STATISTIK (60 detik) ===
Filsuf-1: Makan 23 kali
Filsuf-2: Makan 23 kali
Filsuf-3: Makan 24 kali
Filsuf-4: Makan 22 kali
Filsuf-5: Makan 23 kali

[00] Total berhasil makan: 115 kali
[00] Tidak ada deadlock! (Program berjalan hingga selesai)
[00] Distribusi fairness: BAIK
rafi@DESKTOP-TM81UKJ:~/Tugas5/deadlock_prevention$

```

- Hasil Statistik yang didapat (Solusi A dan Solusi B)

Filsuf	Solusi A	Solusi B
Filsuf-1	25 kali	23 kali
Filsuf-2	23 kali	23 kali
Filsuf-3	22 kali	24 kali
Filsuf-4	21 kali	22 kali
Filsuf-5	22 kali	23 kali
Total Makan	113 kali	115 kali

3.4. Analisis Hasil

- Apakah Program Berjalan Sesuai Ekspetasi?
Ya, Karena
 - Program Deadlock Demonstration berjalan sesuai ekspetasi dengan menyebabkan deadlock / hang.
 - Program Solusi A dan B berjalan untuk mencegah terjadinya deadlock dan berjalan hingga selesai.
- Masalah Yang Ditemukan
 - Deadlock
 - Solusi B mengurangi konkurensi (hanya 4 yang boleh masuk) tetapi memastikan deadlock tidak terjadi.
- Performa Program
 - Efisiensi : Kedua Solusi mencegah terjadinya deadlock dan menunjukkan jumlah makan yang tinggi (113 -115 kali dalam 60 detik).

- Fairness : Solusi A menunjukkan fairness cukup/kurang dan Solusi B menunjukkan fairness BAIK.
- Pelajaran yang didapat
 - Solusi A mengajarkan cara mengubah urutan akses sumber daya
 - Solusi B mengajarkan cara mebatasi jumlah proses yang dapat mencapai kondisi rentan.